

实验四

年级：_____2022_____专业：_____计算机科学与技术_____

姓名：_____姚知言_____学号：_____2211290_____

1 实验内容

1. 若二叉树关键字不重复，根据输入的前序遍历与中序遍历次序构造二叉树，并用层序遍历输出；

2. 根据 1 中构造的二叉树，给定节点值 x (仅描述题目用)，返回节点 x 为根的子树的高度；若 x 节点的父节点为 y (若 x 为根节点不作操作给出提示)，将以 y 为根的子树做如下旋转变换：该 x 节点提升为新的根节点，原根节点 y 变为新根节点 x 的子树，并对原 x 节点的左右子树做如下操作：

2.1 若为右旋转， x 的左子树不变，右子树根节点转换为 y 的左子树

2.2 若为左旋转， x 的右子树不变，左子树根节点转换为 y 的右子树

输出转换后的二叉树；

3. 判断 2 中的结果是否为完全二叉树，若不是则将该树转为完全二叉树并输出。

说明：

(1) 使用链式存储完成题目中的操作

(2) 每问以层序遍历进行输出，输出时将结点的左右孩子结点在括号内输出，若为空则输出#

2 设计思想

该程序的设计思想如下：

定义 int 数组 pre, in, 利用 int 值 pos 和 getchar 将输入的前序中序遍历数组读入。

定义队列类 MQ 和 MQnode, 实现 MQ 的构造, 析构函数, 实现 empty 判空, front 返回首元, push 入队, pop 出队操作。

定义二叉树类 BT 和 BTnode, 实现 BT 的构造, 析构函数, 定义 release 函数递归协助析构函数实现, 定义 addLchild, addRchild 函数为结点添加左右子结点, defineroot 添加根结点。

为完成题目中每一问按层序输出的要求, 定义函数 visit 实现某个结点, 打印 data(lchild data, rchild data) 的功能。定义函数 levelorder, 调用 visit, 借助队列 MQ 的入队, 出队完成层序遍历, 借助 count, countc, countt 来存储输出结点数和空结点数确定换行位置。

(1) 前序中序遍历构造二叉树

定义函数 maketreeforPI, len<=0 弹出, 否则定义 pre[0] 为根结点, 并调用定义的 makeLtreeforPI, makeRtreeforPI 递归实现通过前序中序遍历建树的操作, 并输出。

对该部分的设计思路解释如下:

假设某个结点 x 的左 (右) 子树的前序中序遍历结果范围为原数组的 [start, end] 区间, 由于先序遍历是根-左子树-右子树的顺序, 则他的左 (右) 子节点的 data 为 pre[start], 设该子结点为 y, 即此子树的根结点。

由于中序遍历是左子树-根-右子树的顺序, 则 in 数组在 data 之前的数据为 y 结点 (假设位置为 in[u]) 左子树的遍历结果, 之后的为右子树的遍历结果, 将 in 数组 y 之前的结点向后移动一位, 与 pre 数组对齐, 则 y 结点的左子树的遍历结果为 [start+1, i], 右子树的遍历结果为 [i+1, end], 实现递归。

边界处理: start<end, 则不存在, return, start=end 即只有一个结点, 连接后 return。

最终进行 levelorder 输出。

(2) 返回以结点 x 为根的子树的高度

定义 search 函数, 对于某个数值通过层序遍历寻找此结点, 并返回。

定义 measureheight 函数, 递归查找结点为子树的高度, 思路为以左子树根

结点（如有）为根结点的子树的高度和以右子树根结点（如有）为根结点的子树的高度的较大数值+1，（如都没有为叶子结点，高度为1，也满足）。

定义 height 函数，返回 measureheight (search (x))，即为所求。

（3）左右旋转

构建 turning 函数为此题主函数，定义 searchfather 函数（与 search 函数思路相似，返回目标结点的父结点，增加为根结点返回报错，满足题意）。若目标结点为父结点的左子结点，则右旋转进入 turnright 函数，若为右子结点则左旋进入 turnleft 函数。

对于 turnright 函数与 turnleft 函数（以 turnright 函数为例，二者仅有方向区别），（显然，题意的 x 为 father->Lchild, y 为 father）若 father 为根结点，则根节点改为 father 的 Lchild，否则查找 father 的父结点（searchfather 实现）并将其指向 father 结点的指针改为指向 father 的 Lchild。然后将 x 的右子结点指针指向 father，father 的左子结点指针指向 x 原本的右子结点指针（在上一步操作前用 temp 指针预存）。

最终进行 levelorder 输出。

（4）判断修复完全二叉树

定义 countnode 函数，返回二叉树总结点数，递归方式为左右子树结点数之和+1。

定义 cutnode 函数，在该树非空时，提供 iscut 调用。

定义 iscut 函数通过递归每个结点，自底向上的对超出完全二叉树范围的结点从树分割出来，存入 temp 结点指针数组中待用。

定义 addnode 函数，在该树非空时，提供 isadd 调用。

定义 isadd 函数，对每个结点自顶向下进行递归，对于应有结点但没有的结点弹出队列头的结点连接。

定义 makeperfect 函数，封装，先进行 countnode 和 cutnode 函数，若此时 temp 结点指针数组为空，则为完全二叉树，返回 True，否则返回 False，并执行 addnode 函数。

最终进行 levelorder 输出。

3 程序效果

程序的运行效果，输入及输出的相关要求和具体执行结果如下所示：

3.1 输入

(1) in: [3, 9, 4, 8, 1, 20, 15, 17] [4, 9, 8, 1, 3, 15, 20, 17]

(2) in: 9

3.2 输出

(1) out: 3(9, 20)

9(4, 8) 20(15, 17)

4(#, #) 8(#, 1) 15(#, #) 17(#, #)

1(#, #)

(2) out: 3

9(4, 3)

4(#, #) 3(8, 20)

8(#, 1) 20(15, 17)

1(#, #) 15(#, #) 17(#, #)

(3) out: False

9(4, 3)

4(1, 17) 3(8, 20)

1(15, #) 17(#, #) 8(#, #) 20(#, #)

15(#, #)

4 核心代码

```
#include <iostream>
using namespace std;
template<class T>
class MQ;
template<class T>
class MQnode{
public:
    friend class MQ<T>;
private:
    T data;
    MQnode<T>* next;
```

```

};
template<class T>
class MQ{
public:
    MQ() {first=0;}
    ~MQ();
    bool empty() {if(!first)return true;else return false;}
    T front();
    void push(T x);
    void pop();
private:
    MQnode<T>* first;
};
template<class T>
MQ<T>::~~MQ() {
    auto* temp=first;
    while(first) {
        first=temp->next;
        delete[] temp;
        temp=first;
    }
}
template<class T>
void MQ<T>::push(T x) {
    auto* p=new MQnode<T>;
    p->data=x;p->next=0;
    if(empty()) {first=p;return;}
    auto* q=first;
    while(q->next)q=q->next;
    q->next=p;
}
template<class T>
void MQ<T>::pop() {
    MQnode<T>*p=first->next;
    delete[] first;
    first=p;
}
template<class T>
T MQ<T>::front() {
    if(empty())throw invalid_argument("No item!");
    return first->data;
}
template<class T>
class BT;

```

```

template<class T>
class BTreeNode{
public:
    friend class BT<T>;
private:
    T data;
    BTreeNode<T> *Lchild,*Rchild;
};

template<class T>
class BT{
public:
    BT() {root=0;}
    ~BT();
    void release(BTreeNode<T>* node);
    void addLchild(BTreeNode<T>* father,T x);
    void addRchild(BTreeNode<T>* father,T x);
    void defineroot(T x);
    void maketreeforPI(T* pre,T* in,int len);
    void makeltreeforPI(BTreeNode<T>* father,int start,int end,T* pre,T* in);
    void makeRtreeforPI(BTreeNode<T>* father,int start,int end,T* pre,T* in);
    void visit(BTreeNode<T>* node);
    void levelorder();
    int height(int x);
    int measureheight(BTreeNode<T>* node);
    BTreeNode<T>* search(int x);
    void turning(int x);
    BTreeNode<T>* searchfather(int x);
    void turnright(BTreeNode<T>* father);
    void turnleft(BTreeNode<T>* father);
    int countnode(BTreeNode<T>* base=0);
    void makeperfect();
    void cutnode(MQ<BTreeNode<T>*>& temp,int nodenum);
    void iscut(MQ<BTreeNode<T>*>& temp,BTreeNode<T>* jud,int pos,int nodenum);
    void addnode(MQ<BTreeNode<T>*>& temp,int nodenum);
    void isadd(MQ<BTreeNode<T>*>& temp,BTreeNode<T>* jud,int pos,int nodenum);
private:
    BTreeNode<T>* root;
};

template<class T>
void BT<T>::release(BTreeNode<T>* node) {
    if(node->Lchild)release(node->Lchild);
    if(node->Rchild)release(node->Rchild);
    delete[]node;
}

```

```

template<class T>
BT<T>::~~BT() {release(root);}
template<class T>
void BT<T>::addLchild(BTnode<T>* father, T x) {
    if(father->Lchild!=0)throw invalid_argument("No place!");
    auto* temp=new BTnode<T>;
    temp->data=x;
    temp->Lchild=temp->Rchild=0;
    father->Lchild=temp;
}
template<class T>
void BT<T>::addRchild(BTnode<T>* father, T x) {
    if(father->Rchild!=0)throw invalid_argument("No place!");
    auto* temp=new BTnode<T>;
    temp->data=x;
    temp->Lchild=temp->Rchild=0;
    father->Rchild=temp;
}
template<class T>
void BT<T>::defineroot(T x) {
    if(root!=0)throw invalid_argument("No place!");
    root=new BTnode<T>;
    root->data=x;
    root->Lchild=root->Rchild=0;
}
template<class T>
void BT<T>::makeLtreeforPI(BTnode<T>* father, int start, int end, T* pre, T* in) {
    if(start>end)return;
    addLchild(father, pre[start]);
    if(start==end)return;
    int i;
    for(i=start; i<=end; i++) if(in[i]==pre[start])break;
    for(int j=i; j>start; j--) in[j]=in[j-1];
    makeLtreeforPI(father->Lchild, start+1, i, pre, in);
    makeRtreeforPI(father->Lchild, i+1, end, pre, in);
}
template<class T>
void BT<T>::makeRtreeforPI(BTnode<T>* father, int start, int end, T* pre, T* in) {
    if(start>end)return;
    addRchild(father, pre[start]);
    if(start==end)return;
    int i;
    for(i=start; i<=end; i++) if(in[i]==pre[start])break;
    for(int j=i; j>start; j--) in[j]=in[j-1];
}

```

```

        makeLtreeforPI(father->Rchild, start+1, i, pre, in);
        makeRtreeforPI(father->Rchild, i+1, end, pre, in);
    }
template<class T>
void BT<T>::maketreeforPI(T* pre, T* in, int len) {
    if(len<=0) throw invalid_argument("Invalid input!");
    defineroot(pre[0]);
    int i;
    for(i=0; i<len; i++) if(in[i]==pre[0]) break;
    for(int j=i; j>0; j--) in[j]=in[j-1];
    makeLtreeforPI(root, 1, i, pre, in);
    makeRtreeforPI(root, i+1, len-1, pre, in);
}

template<class T>
void BT<T>::visit(BTnode<T>* node) {
    cout<<node->data<<' (';
    if(node->Lchild) cout<<node->Lchild->data;
    else cout<<' #' ;
    cout<<',';
    if(node->Rchild) cout<<node->Rchild->data;
    else cout<<' #' ;
    cout<<") ";
}

template<class T>
void BT<T>::levelorder() {
    MQ<BTnode<T>*> order;
    int count=0, countc=1, countt=0;
    if(root!=0)
        order.push(root);
    while(!order.empty()) {
        visit(order.front()); count++;
        if(order.front()->Lchild) order.push(order.front()->Lchild);
        else countt++;
        if(order.front()->Rchild) order.push(order.front()->Rchild);
        else countt++;
    }

    if(count==countc) {cout<<' \n'; countc=countc*2+1; count+=countt; countt*=2;}
    order.pop();
}

template<class T>
BTnode<T>* BT<T>::search(int x) {
    MQ<BTnode<T>*> order;
    if(root!=0)

```



```

        order.push(root);
    while(!order.empty()) {
        if(order.front()->data==x) {return order.front();}
        if(order.front()->Lchild)order.push(order.front()->Lchild);
        if(order.front()->Rchild)order.push(order.front()->Rchild);
        order.pop();
    }
    throw invalid_argument("Not found!");
}

template<class T>
BTnode<T>* BT<T>::searchfather(int x) {
    MQ<BTnode<T>*> order;
    if(root->data==x)throw invalid_argument("Can not turn root!");
    if(root!=0)
        order.push(root);
    while(!order.empty()) {
        if(order.front()->Lchild) {
            order.push(order.front()->Lchild);
            if(order.front()->Lchild->data==x)
                return order.front();
        }
        if(order.front()->Rchild) {
            order.push(order.front()->Rchild);
            if(order.front()->Rchild->data==x)
                return order.front();
        }
        order.pop();
    }
    throw invalid_argument("Not found!");
}

template<class T>
int BT<T>::height(int x) {
    return measureheight(search(x));
}

template<class T>
int BT<T>::measureheight(BTnode<T>* node) {
    int ans=1;
    if(node->Lchild)ans=max(ans,measureheight(node->Lchild)+1);
    if(node->Rchild)ans=max(ans,measureheight(node->Rchild)+1);
    return ans;
}

template<class T>
void BT<T>::turnright(BTnode<T>* father) {
    if(root==father)root=father->Lchild;

```

```

    else{
        auto* grandfather=searchfather(father->data);
        if(grandfather->Lchild==father) grandfather->Lchild=father->Lchild;
        else grandfather->Rchild=father->Lchild;
    }
    auto* temp=father->Lchild->Rchild;
    father->Lchild->Rchild=father;
    father->Lchild=temp;
}

template<class T>
void BT<T>::turnleft(BTnode<T>* father) {
    if(root==father) root=father->Rchild;
    else{
        auto* grandfather=searchfather(father->data);
        if(grandfather->Lchild==father) grandfather->Lchild=father->Rchild;
        else grandfather->Rchild=father->Rchild;
    }
    auto* temp=father->Rchild->Lchild;
    father->Rchild->Lchild=father;
    father->Rchild=temp;
}

template<class T>
void BT<T>::turning(int x) {
    auto* father=searchfather(x);
    if(father->Lchild&&father->Lchild->data==x) turnright(father);
    else turnleft(father);
}

template<class T>
int BT<T>::countnode(BTnode<T>* base) {
    if(!base) base=root;
    if(!base) return 0;
    int nodenum=1;
    if(base->Lchild) nodenum+=countnode(base->Lchild);
    if(base->Rchild) nodenum+=countnode(base->Rchild);
    return nodenum;
}

template<class T>
void BT<T>::iscut(MQ<BTnode<T>*>& temp,BTnode<T>* jud,int pos,int nodenum) {
    if(jud->Lchild) {
        iscut(temp, jud->Lchild, pos*2, nodenum);
        if(pos*2>nodenum) {temp.push(jud->Lchild); jud->Lchild=0;}
    }
    if(jud->Rchild) {
        iscut(temp, jud->Rchild, pos*2+1, nodenum);
    }
}

```

```

        if(pos*2+1>nodenum) {temp.push(jud->Rchild);jud->Rchild=0;}
    }
}

template<class T>
void BT<T>::cutnode(MQ<BTnode<T>*>& temp,int nodenum) {
    if(!root)return;
    iscut(temp,root,1,nodenum);
}

template<class T>
void BT<T>::isadd(MQ<BTnode<T>*>& temp,BTnode<T>* jud,int pos,int nodenum) {
    if(pos*2<=nodenum) {
        if(!jud->Lchild){jud->Lchild=temp.front();temp.pop();}
        isadd(temp,jud->Lchild,pos*2,nodenum);
    }
    if(pos*2+1<=nodenum) {
        if(!jud->Rchild){jud->Rchild=temp.front();temp.pop();}
        isadd(temp,jud->Rchild,pos*2+1,nodenum);
    }
}

template<class T>
void BT<T>::addnode(MQ<BTnode<T>*>& temp,int nodenum) {
    if(!root)return;
    isadd(temp,root,1,nodenum);
}

template<class T>
void BT<T>::makeperfect() {
    int nodenum=countnode();
    MQ<BTnode<T>*> temp;
    cutnode(temp,nodenum);
    if(temp.empty())cout<<"True\n";
    else {
        cout<<"False\n";
        addnode(temp,nodenum);
    }
}

int main() {
    int pre[100],in[100],pos=0;
    while(getchar()!='[');
    do cin>>pre[pos++];
    while(getchar()!=']');
    pos=0;
    while(getchar()!='[');
    do cin>>in[pos++];
    while(getchar()!=']');
}

```

```
BT<int> A;
A.maketreeforPI(pre, in, pos);
A.levelorder();
int x;
cin>>x;
cout<<A.height(x)<<' \n';
A.turning(x);
A.levelorder();
A.makeperfect();
A.levelorder();
return 0;
}
```

5 总结

本次实验复习了二叉树的结构,复习了通过先序中序遍历建立二叉树的知识,复习了二叉树高度的概念和计算方法,通过旋转操作复习了二叉树内部操作的实现。复习了完全二叉树的概念和判断方法,并通过构建完全二叉树对二叉树结点连接进行了复习。